APOLLO: SGD-like Memory, AdamW-level Performance

Felipe Vicentin Ying Jin
Training and Deploying Large-Scale Models Poster Project

Zhu et al., MLSys 2025
github.com/PineappleBlowsnow/Course-project-APOLLO-SGD-LIKE-MEMORY-ADAMW-LEVEL-PERFORMANCE

1 Introduction & APOLLO

Training LLMs with AdamW costs **3× parameter memory** due to optimizer states. The update rule is:

$$M_t = \beta_1 M_{t-1} + (1-\beta_1) G_t, \qquad V_t = \beta_2 V_{t-1} + (1-\beta_2) G_t^2$$

$$W_{t+1} = W_t - \eta \cdot \frac{M_t}{\sqrt{V_t+\epsilon}} - \eta\lambda W_t = W_t - \eta \cdot \underbrace{\frac{\tilde{G}_t}{G_t}}_{S} \odot G_t - \eta\lambda W_t \quad \text{(element-wise update)}$$

where $\tilde{G}_t = \frac{M_t}{\sqrt{V_t+\epsilon}}$. Each S_{ij} scales the gradient by its variance to stabilize updates.

To reduce memory, we propose to group parameters by channel j with a shared scaling factor: $s_j = \frac{\|\tilde{G}_t[:,j]\|_2}{\|G_t[:,j]\|_2}$.

APOLLO combines two key ideas:

1. Adaptive scales are shared across a channel or a whole tensor.
2. Statistics for estimating these scales are maintained in a low-rank subspace.

Specifically, we project G_t to rank $r \ll m$ and estimate s_j in that space:

$$R_t = P_t G_t \in \mathbb{R}^{r\times n}, \qquad s_j^R = \frac{\|\tilde{R}_t[:,j]\|_2}{\|R_t[:,j]\|_2} \approx s_j \cdot \sqrt{\frac{r}{n}}$$

Random projection $P_t \sim \mathcal{N}(0, \frac{1}{r})$ is $\mathcal{O}(mnr)$ and provably preserves norms, making it sufficient for approximating s_j.

2 APOLLO Algorithm

Algorithm AdamW with APOLLO/APOLLO-Mini

Input: A weight matrix $\mathbf{W} \in \mathbb{R}^{m\times n}$ with $m \le n$. Step size η, scale factor α, decay rates $\{\beta_1, \beta_2\}$, weight decay λ, rank r, subspace update frequency T.
Initialize: $t \leftarrow 0$
repeat
  # Step 1: Calculate gradient into low rank space.
  $\mathbf{G}_t \in \mathbb{R}^{m\times n} \leftarrow -\nabla_{\mathbf{W}} \phi_t(\mathbf{W}_t)$
  if t mod $T = 0$ **then**
    $\mathbf{P}_t \leftarrow \mathcal{N}_{\text{seed}}(0, 1/r)$
    seed $\leftarrow$ an independent new random seed
  end if
  $\mathbf{R}_t \leftarrow \mathbf{P}_t \mathbf{G}_t$
  # Step 2: Obtain low rank optimization states, $\mathbf{M}_t$, $\mathbf{V}_t$.
  $\mathbf{M}_t^R, \mathbf{V}_t^R \leftarrow \texttt{AdamW}(\mathbf{R}_t, \beta_1, \beta_2, \lambda{=}0)$
  $\widetilde{\mathbf{R}}_t \leftarrow \mathbf{M}_t^R / (\sqrt{\mathbf{V}_t^R} + \epsilon)$
  # Step 3: Obtain approximated gradient scaling factor.
  if APOLLO **then**
    $\mathbf{S} \leftarrow \mathrm{diag}(s_0^R, s_1^R, \ldots, s_m^R) \qquad \left\{ s_i^R = \frac{\|\widetilde{\mathbf{R}}_t[:,j]\|_2}{\|\mathbf{R}_t[:,j]\|_2} \right\}$
  else if APOLLO-Mini **then**
    $\mathbf{S} \leftarrow s^R \qquad \left\{ s^R = \frac{\|\widetilde{\mathbf{R}}_t\|_2}{\|\mathbf{R}_t\|_2} \right\}$
  end if
  # Step 4: Update weight in original space.
  $\mathbf{W}_t \leftarrow \mathbf{W}_{t-1} + \eta \cdot \alpha \cdot \mathbf{G}_t \mathbf{S} - \eta \cdot \lambda \mathbf{W}_{t-1}$
  $t \leftarrow t+1$
until convergence criteria met
return $\mathbf{W}_T$

3 Main Theorem

Combining both moment bounds, the ratio between the low-rank scaling factor s_j^R and the true scaling factor s_j satisfies, with probability $\ge 1-\delta$:

$$\sqrt{\frac{1-\epsilon}{1+\epsilon}} \le \sqrt{\frac{n}{r}} \cdot \frac{s_j^R}{s_j} \le \sqrt{\frac{1+\epsilon}{1-\epsilon}}$$

- $s_j = \|\tilde{G}_t[:,j]\|_2 / \|G_t[:,j]\|_2$: the true channel-wise scaling factor.
- $s_j^R = \|\tilde{R}_t[:,j]\|_2 / \|R_t[:,j]\|_2$: the approximated scaling factor.
- $\sqrt{n/r}$: a deterministic correction factor arising from dimension reduction.
- δ: failure probability; requires $r \ge \frac{8}{\epsilon^2}\log(\frac{2t}{\delta})$, which grows only as $\log t$.

4 Pretaining Models

We focus on **pre-training** to stress the full optimization trajectory and reveal stability differences between optimizers. All experiments were done on Google Colab, with A100 GPUs.

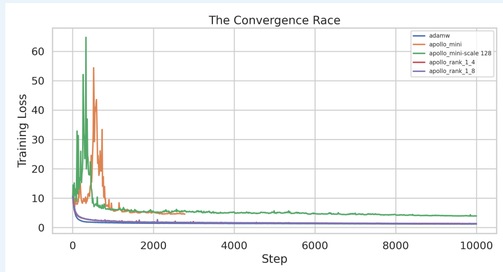
Exp.	Model	Dataset	Optimizers
1	LLaMA-60M	TinyStories	AdamW, APOLLO $r{=}n/4$, $r{=}n/8$, Mini with scale = 16, 128
2	NanoGPT-40M	TinyStories	AdamW, APOLLO $r{=}n/4$, channel-wise, group-wise k = 2, 4
3	NanoGPT-130M	WikiText	AdamW, channel-wise structured scaling

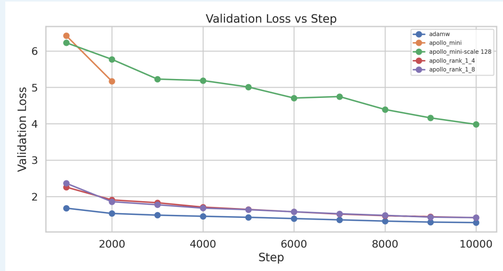
Metrics: Validation perplexity, optimizer-state memory.

5 Partial Success: LLaMA-60M + TinyStories:

- **Core Validation:** APOLLO (rank $n/4, n/8$) remains stable and closely tracks AdamW's performance.
- **APOLLO-Mini Failure:** Tensor-wise scaling (rank-1) is insufficient for early-stage stability, leading to severe loss spikes and poor convergence (53.66 vs. 3.61 PPL).
- **Rank Robustness:** Rank $n/8$ performance matches rank $n/4$ almost exactly, proving that structured scaling is highly robust to extreme rank reduction.

Optimizer	Perplexity↓	Opt. Mem.
AdamW	**3.61**	0.433 GB
APOLLO $r{=}n/4$	4.13	0.108 GB
APOLLO $r{=}n/8$	4.13	0.054 GB
APOLLO-Mini	53.66	0.001 GB



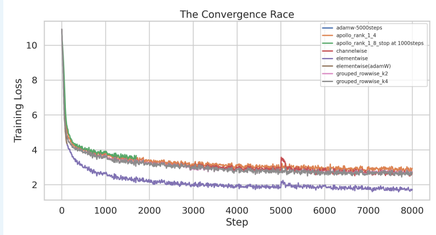


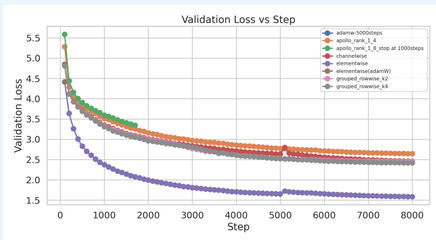
6 Sanity Check Fails on NanoGPT-40M + TinyStories

- **Channel-wise scaling** underperforms significantly compared to AdamW (Perplexity: 14.10 vs. 5.23).
- Even **minimal grouping** ($K = 2, 4$) fails to match AdamW, proving that coarse scaling discards vital element-wise optimization signals.
- While **LLaMA** benefits from structured adaptivity, **NanoGPT** does not, suggesting sensitivity to specific architectural choices.

Optimizer	Perplexity (↓)
AdamW (Element-wise)	**5.23**
Group-wise $K = 4$	11.22
Group-wise $K = 2$	11.59
Channel-wise	14.10

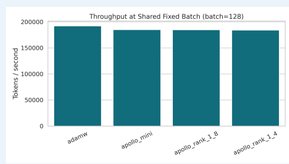
Note: NanoGPT-130M + WikiText results (bottom plot) confirm the element-wise advantage.



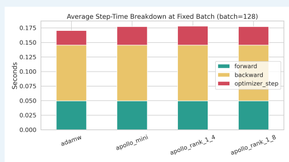


7 Memory Analysis & Scaling Ratio Diagnostic on Llama experiments

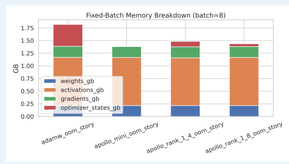
- **Memory Efficiency:** Optimizer-state memory is reduced **proportionally to the applied rank**. OOM stress tests confirm APOLLO supports larger maximum batch sizes than AdamW.
- **Activation Bottleneck:** The improvement in max batch size is modest at large scales, as **activations** become the primary memory bottleneck.
- **Throughput & Step-time:** Throughput is comparable to AdamW. On single-GPU runs, performance is often **compute-bound** rather than memory-bound, meaning lower memory overhead does not always yield higher tokens/sec.
- **Fail sanity check on correction factor:** The scaling ratio (s_j^R/s_j) consistently stays **below theoretical upper bounds**. The discrepancies relative to the original paper are likely attributed to our reduced experimental setup and smaller model scale.



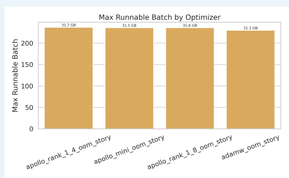
(a) Throughput



(b) Average step-time



(c) Memory breakdown



(d) Maximum batches

Scaling factor

8 Conclusion

Validated:

- On LLaMA-60M, APOLLO $r{=}n/4$ and $r{=}n/8$ proportionally (to rank) reduce optimizer-state memory while staying close to AdamW perplexity
- Rank compression is robust: halving rank has negligible perplexity impact

Beyond the paper:

- Structured scaling fails on NanoGPT-40M and NanoGPT-130M across two datasets
- APOLLO-Mini is unstable: rank-1 is too coarse without careful retuning
- Activation memory limits practical throughput gains at small scale

APOLLO is a conditional win: compelling when the architecture makes structured adaptive scaling compressible (LLaMA), much less so otherwise (NanoGPT). This theory/practice gap is exactly worth reporting.